

```
"""
```

```
GERADOR DE ESCALAS MÉDICAS – Streamlit App
```

```
Autor: gerado para junia.sleite@gmail.com
```

```
Versão: 1.0
```

```
"""
```

```
import streamlit as st
```

```
import pandas as pd
```

```
from datetime import date, timedelta
```

```
from collections import defaultdict
```

```
import io
```

```
import openpyxl
```

```
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
```

```
import json
```

```
import pickle
```

```
import os
```

```
# _____
```

```
# CONFIGURAÇÃO DA PÁGINA
```

```
# _____
```

```
st.set_page_config(  
    page_title="Gerador de Escalas Médicas",  
    page_icon="🏥",  
    layout="wide",  
    initial_sidebar_state="expanded"  
)
```

```
st.markdown("""
```

```
<style>
```

```
    .main-title { font-size: 2rem; font-weight: bold; color: #1F4E79; }
```

```
    .section-title { font-size: 1.2rem; font-weight: bold; color: #2874A6;
```

```
        border-bottom: 2px solid #2874A6; padding-bottom: 4px; margin-bottom: 10px; }
```

```
    .info-box { background: #EBF5FB; border-left: 4px solid #2874A6;
```

```
        padding: 10px; border-radius: 4px; margin: 8px 0; }
```

```
    .warn-box { background: #FEF9E7; border-left: 4px solid #F39C12;
```

```
        padding: 10px; border-radius: 4px; margin: 8px 0; }
```

```
    .ok-box { background: #EAFAF1; border-left: 4px solid #27AE60;
```

```
        padding: 10px; border-radius: 4px; margin: 8px 0; }
```

```
    .err-box { background: #FDEDEC; border-left: 4px solid #E74C3C;
```

```
        padding: 10px; border-radius: 4px; margin: 8px 0; }
```

```
</style>
```

```
""", unsafe_allow_html=True)
```

```
# _____
```

[illegible]

```

        for k in list(st.session_state.keys()):
            del st.session_state[k]
        st.rerun()

# Salvar/carregar configuração
st.markdown("### 💾 Salvar Configuração")
if st.session_state.step >= 5:
    cfg_export = {
        "config": st.session_state.config,
        "locais": st.session_state.locais,
        "regras": st.session_state.regras,
        "ch_limites": st.session_state.ch_limites,
        "fds_config": st.session_state.fds_config,
    }
    st.download_button(
        "⬇️ Salvar regras (JSON)",
        data=json.dumps(cfg_export, ensure_ascii=False, indent=2),
        file_name=f"regras_{st.session_state.config.get('especialidade', 'esca')}.json",
        mime="application/json"
    )

uploaded_cfg = st.file_uploader("📁 Carregar regras salvas", type="json")
if uploaded_cfg:
    try:
        loaded = json.load(uploaded_cfg)
        st.session_state.config = loaded.get("config", {})
        st.session_state.locais = loaded.get("locais", [])
        st.session_state.regras = loaded.get("regras", {})
        st.session_state.ch_limites = loaded.get("ch_limites", {})
        st.session_state.fds_config = loaded.get("fds_config", [])
        st.success("Regras carregadas! Vá para o passo 2 para adicionar os al")
    except Exception as e:
        st.error(f"Erro ao carregar: {e}")

# -----
# PASSO 1 – IDENTIFICAÇÃO
# -----
if st.session_state.step == 1:
    st.markdown('<p class="main-title">🏥 Gerador de Escalas Médicas</p>', unsafe_allow_html=True)
    st.markdown('<p class="section-title">Passo 1 – Identificação da Escala</p>', unsafe_allow_html=True)

    col1, col2 = st.columns(2)
    with col1:
        especialidade = st.text_input("Especialidade *", placeholder="ex: Clínica")
        ano = st.selectbox("Ano do curso *", ["3º Ano", "4º Ano", "5º Ano", "6º Ano"])
        grupo = st.text_input("Grupo *", placeholder="ex: Grupo A")
    with col2:

```

```

turma = st.text_input("Turma *", placeholder="ex: T6")
data_inicio = st.date_input("Data de início (segunda-feira) *", value=dat
n_semanas = st.number_input("Número de semanas *", min_value=1, max_value

# Calcular e mostrar data de fim
data_fim = data_inicio + timedelta(weeks=n_semanas) - timedelta(days=1)
st.markdown(f'<div class="info-box">17 Período: <b>{data_inicio.strftime("%d/

# Verificar se início é segunda
if data_inicio.weekday() != 0:
    st.markdown('<div class="warn-box">⚠️ A data de início deve ser uma segun

st.markdown("---")
if st.button("Próximo →", type="primary", disabled=not (especialidade and gru
    st.session_state.config = {
        "especialidade": especialidade,
        "ano": ano,
        "grupo": grupo,
        "turma": turma,
        "data_inicio": data_inicio.strftime("%d/%m/%Y"),
        "data_fim": data_fim.strftime("%d/%m/%Y"),
        "n_semanas": int(n_semanas),
    }
    st.session_state.step = 2
    st.rerun()

# -----
# PASSO 2 – ALUNOS E SUBGRUPOS
# -----
elif st.session_state.step == 2:
    cfg = st.session_state.config
    st.markdown(f'<p class="main-title">{cfg["especialidade"]} – {cfg["grupo"]} /
    st.markdown(f'<p class="section-title">Passo 2 – Alunos e Subgrupos</p>', unsa

    n_sgs = st.number_input("Quantos subgrupos?", min_value=1, max_value=10, valu

    st.markdown("### Adicionar alunos")
    st.markdown('<div class="info-box">💡 Cole os alunos linha a linha no formatc

    # Área de texto para colar alunos em massa
    texto_alunos = st.text_area(
        "Cole os alunos aqui (um por linha: Nome | RA | SG)",
        height=300,
        placeholder="Larissa Rosset Fávero Charneski | 220061039 | 1\nJúlia da Si
    )

    alunos_parsed = []

```

```

erros_parse = []
if texto_alunos.strip():
    for i, linha in enumerate(texto_alunos.strip().split("\n")):
        linha = linha.strip()
        if not linha: continue
        partes = [p.strip() for p in linha.split("|")]
        if len(partes) != 3:
            erros_parse.append(f"Linha {i+1}: formato inválido → '{linha}'")
        else:
            try:
                nome, ra, sg = partes
                sg_int = int(sg)
                if sg_int < 1 or sg_int > n_sgs:
                    erros_parse.append(f"Linha {i+1}: SG{sg_int} inválido (má
            except ValueError:
                erros_parse.append(f"Linha {i+1}: SG deve ser número → '{linh

if erros_parse:
    for e in erros_parse:
        st.markdown(f'<div class="err-box">✖ {e}</div>', unsafe_allow_html=True)

if alunos_parsed:
    # Mostrar resumo por SG
    df = pd.DataFrame(alunos_parsed)
    st.markdown(f'<div class="ok-box">✓ {len(alunos_parsed)} alunos reconheci

    for sg in sorted(df["sg"].unique()):
        alunos_sg = df[df["sg"] == sg]
        with st.expander(f"SG{sg} - {len(alunos_sg)} alunos"):
            st.dataframe(alunos_sg[["nome", "ra"]].reset_index(drop=True), us

    # Resumo de SGs
    resumo = df.groupby("sg").size().reset_index(columns=["sg", "n"])
    resumo.columns = ["Subgrupo", "Alunos"]
    resumo["Subgrupo"] = "SG" + resumo["Subgrupo"].astype(str)
    st.dataframe(resumo, use_container_width=True)

st.markdown("---")
col1, col2 = st.columns(2)
with col1:
    if st.button("← Voltar"):
        st.session_state.step = 1
        st.rerun()
with col2:
    if st.button("Próximo →", type="primary",

```

```

        disabled=len(alunos_parsed) == 0 or len(erros_parse) > 0):
    st.session_state.alunos = alunos_parsed
    # Organizar por SG
    sgs = defaultdict(list)
    for a in alunos_parsed:
        sgs[a["sg"]].append((a["nome"], a["ra"]))
    st.session_state.subgrupos = dict(sgs)
    st.session_state.config["n_sgs"] = n_sgs
    st.session_state.step = 3
    st.rerun()

# -----
# PASSO 3 – LOCAIS E RODÍZIO
# -----

elif st.session_state.step == 3:
    cfg = st.session_state.config
    sgs = st.session_state.subgrupos
    n_sem = cfg["n_semanas"]

    st.markdown(f'<p class="main-title">{cfg["especialidade"]} – {cfg["grupo"]} /
    st.markdown('<p class="section-title">Passo 3 – Locais de Rodízio</p>', unsafe_allow_html=True)

    # Definir locais
    st.markdown("### 1. Quais são os locais de estágio?")
    n_locais = st.number_input("Quantos locais?", min_value=1, max_value=8, value=1)

    locais = []
    cols = st.columns(min(n_locais, 4))
    for i in range(int(n_locais)):
        with cols[i % 4]:
            nome_local = st.text_input(f"Local {i+1}", key=f"local_{i}",
                                       placeholder=["Enfermaria", "PS", "Ambulatório"])
            abrev = st.text_input(f"Abreviação {i+1}", key=f"abrev_{i}",
                                  placeholder=["Enf", "PS", "Amb", "UTI", "CC", "Mat"],
                                  max_chars=6)
            if nome_local and abrev:
                locais.append({"nome": nome_local, "abrev": abrev})

    if len(locais) == int(n_locais):
        st.markdown("---")
        st.markdown("### 2. Agrupamento de subgrupos")
        st.markdown('<div class="info-box">💡 Subgrupos que ficam JUNTOS no mesmo semestre')

        # Opção de agrupar SGs em pares
        usar_pares = st.checkbox("Os subgrupos são agrupados em pares/trios?", value=True)

        pares = {}

```

```

if usar_pares:
    n_pares = st.number_input("Quantos grupos/pares?", min_value=1,
                               max_value=len(sgs), value=min(3, len(sgs)))
    sgs_disponiveis = sorted(sgs.keys())
    for p in range(int(n_pares)):
        st.markdown(f"***Par/Grupo {p+1}:***")
        sgs_par = st.multiselect(
            f"Subgrupos do Grupo {p+1}",
            options=[f"SG{s}" for s in sgs_disponiveis],
            key=f"par_{p}",
            default=[f"SG{sgs_disponiveis[p*2]}", f"SG{sgs_disponiveis[p*2+1]}"]
        )
        if sgs_par:
            pares[f"Par{p+1}"] = [int(s.replace("SG","")) for s in sgs_par]
    else:
        # Cada SG é independente
        for sg in sorted(sgs.keys()):
            pares[f"SG{sg}"] = [sg]

st.markdown("---")
st.markdown("### 3. Tabela de Rodízio")
st.markdown('<div class="info-box">💡 Para cada grupo/par, defina qual lc

rodizio = {}
if pares:
    # Criar tabela de rodízio
    opcoes_local = [l["nome"] for l in locais]
    rodizio_data = {}

    for par_nome, par_sgs in pares.items():
        n_alunos_par = sum(len(sgs[s]) for s in par_sgs if s in sgs)
        st.markdown(f"***{par_nome}*** (SG{' '+SG'.join(str(s) for s in par_sgs)

    cols_sem = st.columns(min(n_sem, 8))
    semanas_local = []
    for sem in range(1, n_sem+1):
        with cols_sem[(sem-1) % 8]:
            local_sem = st.selectbox(
                f"S{sem}",
                opcoes_local,
                key=f"rod_{par_nome}_s{sem}",
                index=(list(pares.keys()).index(par_nome)) % len(opcoes_local)
            )
            semanas_local.append(local_sem)
    rodizio_data[par_nome] = {
        "sgs": par_sgs,
        "semanas": semanas_local

```

```

    }

    # Montar schedule: sg → semana → local
    schedule = {}
    for par_nome, par_data in rodizio_data.items():
        for sg in par_data["sgs"]:
            schedule[sg] = {}
            for sem_idx, local_nome in enumerate(par_data["semanas"]):
                sem = sem_idx + 1
                # Encontrar abrev do local
                abrev = next((l["abrev"] for l in locais if l["nome"] == local_nome))
                schedule[sg][sem] = abrev
    rodizio = {"pares": rodizio_data, "schedule": schedule, "pares_def": pares_def}

    st.markdown("---")
    col1, col2 = st.columns(2)
    with col1:
        if st.button("← Voltar"):
            st.session_state.step = 2
            st.rerun()
    with col2:
        if st.button("Próximo →", type="primary", disabled=not rodizio):
            st.session_state.locais = locais
            st.session_state.rodizio = rodizio
            st.session_state.step = 4
            st.rerun()

# -----
# PASSO 4 – REGRAS POR LOCAL
# -----

elif st.session_state.step == 4:
    cfg = st.session_state.config
    locais = st.session_state.locais

    st.markdown(f'<p class="main-title">{cfg["especialidade"]} – {cfg["grupo"]} /
    st.markdown(f'<p class="section-title">Passo 4 – Regras por Local</p>', unsafe_allow_html=True)

    st.markdown(f'<div class="info-box">🔧 Configure as regras de cada local. Esta
    st.markdown(f'</div>', unsafe_allow_html=True)

    regras = {}
    for local in locais:
        nome = local["nome"]
        abrev = local["abrev"]
        with st.expander(f"🏠 {nome} ({abrev})", expanded=True):
            st.markdown("***MANHÃ***")
            c1, c2, c3 = st.columns(3)
            with c1:

```



```

        h_manha_inicio = st.text_input("Início manhã", value="07:00", key
        h_manha_fim = st.text_input("Fim manhã", value="13:00", key=f"{ab
with c2:
        h_manha = st.number_input("Horas manhã", value=6, min_value=1, ma
        todos_manha = st.checkbox("Todos fazem manhã?", value=True, key=f
with c3:
        qui_manha = st.selectbox("Quinta manhã", ["Normal (igual)", "Sem
        ter_manha = st.selectbox("Terça manhã", ["Normal (igual)", "Horár

st.markdown("***TARDE**")
c1, c2, c3 = st.columns(3)
with c1:
        tem_tarde = st.checkbox("Tem tarde?", value=True, key=f"{abrev}_t
        h_tarde_inicio = st.text_input("Início tarde", value="12:00", key
        h_tarde_fim = st.text_input("Fim tarde", value="18:00", key=f"{ab
with c2:
        h_tarde = st.number_input("Horas tarde normal", value=6, min_valu
        n_tarde = st.number_input("Slots tarde/dia", value=3, min_value=1
                                help="Quantos alunos fazem tarde por di
with c3:
        tem_tarde_red = st.checkbox("Tem slot reduzido?", value=True, key
                                help="Ex: 1 aluno sai mais cedo (12-
        if tem_tarde_red:
            h_tarde_red_inicio = st.text_input("Início reduzido", value="
            h_tarde_red_fim = st.text_input("Fim reduzido", value="16:00"
            h_tarde_red = st.number_input("Horas reduzido", value=4, min_

st.markdown("***QUINTA E TERÇA**")
c1, c2 = st.columns(2)
with c1:
        qui_tarde = st.selectbox("Quinta tarde",
                                ["Sem tarde (ENAMED/reunião)", "Tarde normal", "Tarde diferen
                                key=f"{abrev}_qt")
with c2:
        ter_tarde = st.selectbox("Terça tarde",
                                ["Tarde reduzida (12-16h)", "Tarde normal", "Sem tarde"],
                                key=f"{abrev}_tt2")
        if ter_tarde == "Tarde reduzida (12-16h)":
            ter_h_tarde = st.number_input("Horas terça tarde", value=4, m
            ter_todos = st.checkbox("Todos fazem tarde terça?", value=Tru
                                help="Ou só o subgrupo rotativo?")

st.markdown("***FINAL DE SEMANA**")
c1, c2 = st.columns(2)
with c1:
        tem_fds = st.checkbox("Tem plantão FDS?", value=True, key=f"{abre
        if tem_fds:

```

```

        fds_quem = st.selectbox("Quem faz FDS?",
                                ["Alunos deste local", "Alunos de outro local"],
                                key=f"{abrev}_fdsq")
        if fds_quem == "Alunos de outro local":
            fds_local_origem = st.text_input("De qual local?",
                                              placeholder="ex: Ambulatório", key=f"{abrev}_fdso")
with c2:
    if tem_fds:
        fds_n_manha = st.number_input("Alunos manhã FDS", value=1, mi
        fds_n_tarde = st.number_input("Alunos tarde FDS", value=0, mi
        fds_h_manha = st.number_input("Horas manhã FDS", value=5, min
        fds_h_tarde = st.number_input("Horas tarde FDS", value=6, min
        fds_max_por_aluno = st.number_input("Máx FDS por aluno no blo

# Montar regras deste local
r = {
    "manha": {
        "inicio": h_manha_inicio, "fim": h_manha_fim, "horas": int(h_
        "todos": todos_manha,
        "quinta": qui_manha, "terca": ter_manha
    },
    "tarde": {
        "tem": tem_tarde,
        "inicio": h_tarde_inicio if tem_tarde else "",
        "fim": h_tarde_fim if tem_tarde else "",
        "horas": int(h_tarde) if tem_tarde else 0,
        "slots": int(n_tarde) if tem_tarde else 0,
        "reduzido": tem_tarde_red if tem_tarde else False,
        "red_inicio": h_tarde_red_inicio if (tem_tarde and tem_tarde_
        "red_fim": h_tarde_red_fim if (tem_tarde and tem_tarde_red) e
        "red_horas": int(h_tarde_red) if (tem_tarde and tem_tarde_red
        "quinta": qui_tarde if tem_tarde else "Sem tarde",
        "terca": ter_tarde if tem_tarde else "Sem tarde",
        "terca_horas": int(ter_h_tarde) if (tem_tarde and ter_tarde =
        "terca_todos": ter_todos if (tem_tarde and ter_tarde == "Tard
    },
    "fds": {
        "tem": tem_fds,
        "quem": fds_quem if tem_fds else "Alunos deste local",
        "local_origem": fds_local_origem if (tem_fds and fds_quem ==
        "n_manha": int(fds_n_manha) if tem_fds else 0,
        "n_tarde": int(fds_n_tarde) if tem_fds else 0,
        "h_manha": int(fds_h_manha) if tem_fds else 0,
        "h_tarde": int(fds_h_tarde) if (tem_fds and fds_n_tarde > 0)
        "max_por_aluno": int(fds_max_por_aluno) if tem_fds else 0,
    }
}

```

```

        regras[abrev] = r

st.markdown("---")
col1, col2 = st.columns(2)
with col1:
    if st.button("← Voltar"):
        st.session_state.step = 3
        st.rerun()
with col2:
    if st.button("Próximo →", type="primary"):
        st.session_state.regras = regras
        st.session_state.step = 5
        st.rerun()

# -----
# PASSO 5 – LIMITES DE CH E FDS
# -----

elif st.session_state.step == 5:
    cfg = st.session_state.config
    locais = st.session_state.locais

    st.markdown(f'<p class="main-title">{cfg["especialidade"]} – {cfg["grupo"]} /
    st.markdown('<p class="section-title">Passo 5 – Limites de CH e Distribuição

    st.markdown("### Limites de Carga Horária Semanal")
    st.markdown('<div class="info-box">💡 Defina o máximo de horas semanais aceit

    ch_limites = {}
    cols = st.columns(len(locais))
    for i, local in enumerate(locais):
        with cols[i]:
            abrev = local["abrev"]
            st.markdown(f"***{local['nome']}***")
            ch_pad = st.number_input(f"CH padrão (h)", value=40, min_value=20, m
            ch_fds = st.number_input(f"CH com FDS (h)", value=42, min_value=20, m
                                help="Limite aceito quando aluno fez plantã
            ch_abs = st.number_input(f"CH absoluta máx (h)", value=43, min_value=
                                help="Nunca ultrapassar este valor, mesmo e
            ch_limites[abrev] = {"padrao": int(ch_pad), "com_fds": int(ch_fds), "

    st.markdown("---")
    st.markdown("### Quando há conflito entre cobertura e CH:")
    conflito = st.radio(
        "O que fazer quando garantir cobertura mínima exige que alguém passe do l
        ["Aceitar CH maior (até o limite absoluto)", "Manter CH e deixar cobertur
        "Me avisar e eu decido caso a caso"],
        index=0

```

```

)

st.markdown("---")
st.markdown("### Distribuição de Plantões FDS")
st.markdown('<div class="info-box">💡 O sistema vai distribuir os plantões de

meta_fds = st.selectbox(
    "Meta de distribuição FDS",
    ["Mais homogêneo possível (diferença máx de 1 entre alunos do mesmo bloco",
    "Estritamente igual (pode deixar alguns sem plantão)",
    "Livre (sem meta específica)"],
    index=0
)

st.markdown("---")
col1, col2 = st.columns(2)
with col1:
    if st.button("← Voltar"):
        st.session_state.step = 4
        st.rerun()
with col2:
    if st.button("Próximo: Validar e Gerar →", type="primary"):
        st.session_state.ch_limites = ch_limites
        st.session_state.config["conflito_ch"] = conflito
        st.session_state.config["meta_fds"] = meta_fds
        st.session_state.step = 6
        st.rerun()

# -----
# PASSO 6 – VALIDAÇÃO
# -----
elif st.session_state.step == 6:
    cfg = st.session_state.config
    alunos = st.session_state.alunos
    sgs = st.session_state.subgrupos
    locais = st.session_state.locais
    rodizio = st.session_state.rodizio
    regras = st.session_state.regras
    ch_limites = st.session_state.ch_limites

    st.markdown(f'<p class="main-title">{cfg["especialidade"]} – {cfg["grupo"]} /
    st.markdown('<p class="section-title">Passo 6 – Validação da Configuração</p>

    erros = []
    avisos = []

    # Verificar cobertura matemática

```

```

n_sem = cfg["n_semanas"]
schedule = rodizio.get("schedule", {})
pares_def = rodizio.get("pares_def", {})

for par_nome, par_sgs in pares_def.items():
    n_al = sum(len(sgs.get(s, [])) for s in par_sgs)
    semanas_par = rodizio["pares"][par_nome]["semanas"]

    # Para cada local que esse par visita, verificar viabilidade de CH
    for sem_idx, local_nome in enumerate(semanas_par):
        abbrev = next((l["abbrev"] for l in locais if l["nome"] == local_nome),
            if abbrev not in regras: continue
            r = regras[abbrev]
            if not r["tarde"]["tem"]: continue

        # Calcular CH teórica
        h_manha = r["manha"]["horas"]
        h_tarde_norm = r["tarde"]["horas"]
        slots_tarde = r["tarde"]["slots"]
        h_qui_m = h_manha if r["manha"]["quinta"] == "Normal (igual)" else 0
        h_ter_t = r["tarde"]["terca_horas"]

        # CH manhã por aluno (5 dias: seg,ter,qua,qui,sex)
        ch_m = h_manha * 4 + h_qui_m # seg,ter,qua,sex + qui
        # CH tarde estimada por aluno: slots/dia * 4 dias / n_alunos * horas
        ch_t_est = (slots_tarde * 4 * h_tarde_norm) / n_al if n_al > 0 else 0
        ch_total_est = ch_m + ch_t_est

        lim = ch_limites.get(abbrev, {}).get("padrao", 40)
        lim_abs = ch_limites.get(abbrev, {}).get("absoluto", 43)

        if ch_total_est > lim_abs + 2:
            erros.append(f"⚠️ {par_nome} no {local_nome} (S{sem_idx+1}): CH e
        elif ch_total_est > lim:
            avisos.append(f"🚦 {par_nome} no {local_nome}: CH estimada {ch_tc

# Verificar se todos os SGs têm rodizio completo
for sg in sgs.keys():
    if sg not in schedule:
        erros.append(f"SG{sg} não tem rodizio definido")
    elif len(schedule[sg]) < n_sem:
        erros.append(f"SG{sg} tem rodizio para {len(schedule[sg])} semanas, m

# Verificar datas
di_parts = cfg["data_inicio"].split("/")
data_inicio = date(int(di_parts[2]), int(di_parts[1]), int(di_parts[0]))
if data_inicio.weekday() != 0:

```

```

        erros.append("Data de início não é segunda-feira!")

# Mostrar resultado
st.markdown("### Resumo da Configuração")
col1, col2, col3 = st.columns(3)
with col1:
    st.metric("Total de alunos", len(alunos))
    st.metric("Subgrupos", len(sgs))
with col2:
    st.metric("Semanas", n_sem)
    st.metric("Locais", len(locais))
with col3:
    st.metric("Erros", len(erros))
    st.metric("Avisos", len(avisos))

if erros:
    st.markdown("### ❌ Erros (precisam ser corrigidos)")
    for e in erros:
        st.markdown(f'<div class="err-box">{e}</div>', unsafe_allow_html=True)
else:
    st.markdown(f'<div class="ok-box">✅ Configuração válida – sem erros críticos</div>')

if avisos:
    st.markdown("### ⚠️ Avisos (o sistema vai lidar automaticamente)")
    for a in avisos:
        st.markdown(f'<div class="warn-box">{a}</div>', unsafe_allow_html=True)

# Mostrar tabela de rodízio
st.markdown("### Tabela de Rodízio")
rodizio_table = {}
for par_nome, par_data in rodizio["pares"].items():
    par_sgs = par_data["sgs"]
    label = f"{par_nome} (SG{'+'.join(str(s) for s in par_sgs)})"
    rodizio_table[label] = par_data["semanas"]

df_rod = pd.DataFrame(rodizio_table).T
df_rod.columns = [f"S{i+1}" for i in range(len(df_rod.columns))]
st.dataframe(df_rod, use_container_width=True)

st.markdown("---")
col1, col2 = st.columns(2)
with col1:
    if st.button("← Voltar e corrigir"):
        st.session_state.step = 5
        st.rerun()
with col2:
    if not erros:

```

```

if st.button("🚀 GERAR ESCALA", type="primary"):
    with st.spinner("Gerando escala... isso pode levar alguns segundo
    try:
        from gerador_escala import gerar_escala
        resultado = gerar_escala(
            config=config,
            alunos=alunos,
            subgrupos=sgs,
            locais=locais,
            rodizio=rodizio,
            regras=regras,
            ch_limites=ch_limites
        )
        st.session_state.alloc = resultado
        st.session_state.step = 7
        st.rerun()
    except Exception as e:
        st.error(f"Erro na geração: {e}")
        import traceback
        st.code(traceback.format_exc())

# -----
# PASSO 7 - DOWNLOAD
# -----

elif st.session_state.step == 7:
    cfg = st.session_state.config
    resultado = st.session_state.alloc

    st.markdown(f'<p class="main-title">✅ Escala Gerada!</p>', unsafe_allow_html)
    st.markdown(f'<div class="ok-box">🎉 Escala de <b>{cfg["especialidade"]}</b></div>')

    if resultado:
        col1, col2, col3 = st.columns(3)
        with col1:
            if "excel_bytes" in resultado:
                st.download_button(
                    "📄 Baixar Excel (.xlsx)",
                    data=resultado["excel_bytes"],
                    file_name=f"Escala_{cfg['especialidade'].replace(' ', '_')}.xlsx",
                    mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
                    use_container_width=True
                )
        with col2:
            if "csv_bytes" in resultado:
                st.download_button(
                    "📄 Baixar CSV",
                    data=resultado["csv_bytes"],

```

```

        file_name=f"Escala_{cfg['especialidade'].replace(' ','_')}_c
        mime="text/csv",
        use_container_width=True
    )
with col3:
    if "auditoria" in resultado:
        st.download_button(
            " Auditoria: zero erros</div>',
            else:
                st.markdown(f'<div class="warn-box"> Auditoria: {m["erros_audit

st.markdown("---")
col1, col2 = st.columns(2)
with col1:
    if st.button("
```